Procesos de Desarrollo de Software

CLASE 11 – PATRONES DE DISEÑO

Agenda de la clase

- State

State

Es un patrón de **Comportamiento.**

¿Qué hace?

- Genera una abstracción (una clase) por cada posible estado que pueda tener un objeto.
- Define transiciones entre los posibles estados.

Se **sugiere** su utilización **cuando**:

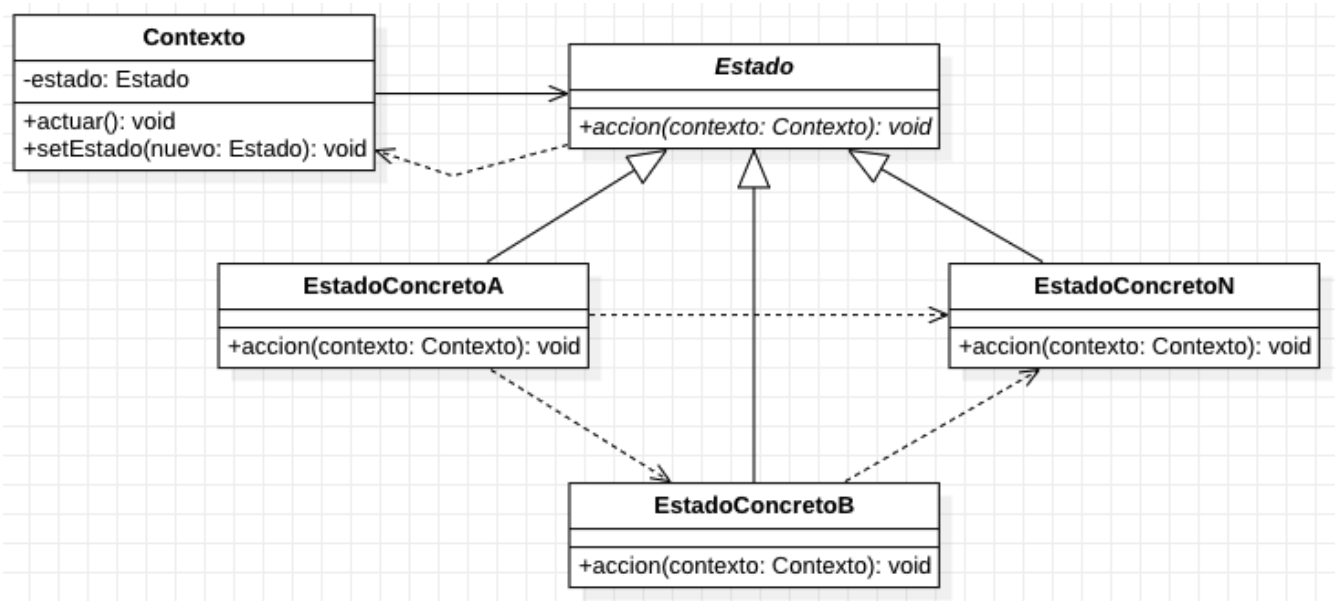
- El comportamiento de un objeto depende de su estado y este mismo puede variar en tiempo de ejecución.
- Un método está lleno de sentencias condicionales que dependen del estado del objeto. Estos estados suelen estar representados por varios atributos de distintos tipos: primitivos (boolean, int, etc.) o por enumerados (enum).

Componentes

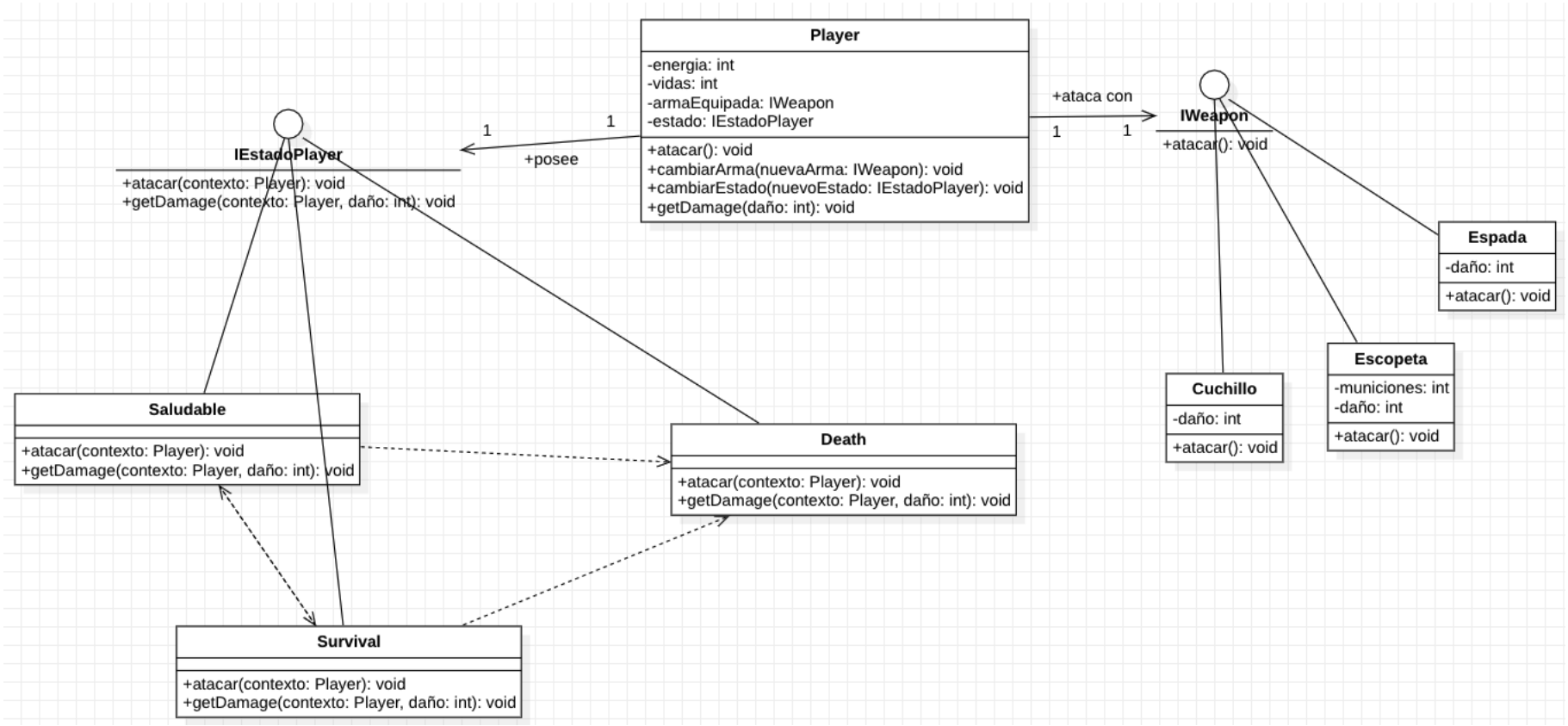
- **Interface (o clase abstracta) State**: Define las firmas de los métodos que dependen del estado del objeto principal.
- **Clases de estados concretas**: Clases que implementan la interface State (o que heredan de ella, si ésta fuera clase abstracta), es decir, que tienen la implementación real de los métodos. Son los estados posibles del objeto principal.
- **Contexto**: Clase que tiene referencia a la interface/clase abstracta State, cuyos objetos van a delegar la responsabilidad de resolución de algunos problemas en el estado. Estos objetos utilizarán de forma polimórfica a los estados.

Diagrama UML

- Beneficios:
  - Mayor mantenibilidad debido a que el comportamiento por cada estado es fácilmente localizable.
  - Extensibilidad para incorporar nuevos estados (nuevos comportamientos).
  - Mayor cohesión a la clase Contexto
- Code smells que soluciona/evita de forma directa:
  - Métodos largos
  - Clase Dios



Ejemplo


Player
-energia: int
-vidas: int
-armaEquipada: IWeapon
-estado: IEstadoPlayer
+atacar(): void
+cambiarArma(nuevaArma: IWeapon): void
+cambiarEstado(nuevoEstado: IEstadoPlayer): void
+getDamage(daño: int): void
1
1
+posee
+ataca con
1
1
IEstadoPlayer
+atacar(contexto: Player): void
+getDamage(contexto: Player, daño: int): void
IWeapon
+atacar(): void
Espada
-daño: int
+atacar(): void
Escopeta
-municiones: int
-daño: int
+atacar(): void
Cuchillo
-daño: int
+atacar(): void
Saludable
+atacar(contexto: Player): void
+getDamage(contexto: Player, daño: int): void
Death
+atacar(contexto: Player): void
+getDamage(contexto: Player, daño: int): void
Survival
+atacar(contexto: Player): void
+getDamage(contexto: Player, daño: int): void

Práctica

- Realizar el ejercicio de aplicación de patrones "**For Sale**"
- Realizar el ejercicio de aplicación de patrones "**Flip Flop**"

Magic Café ☕ ✨

- De manejar estados con if/else
  → a encapsular comportamiento por estado
- MagicCafe ya tiene:
  - Hibernate/JPA
  - Clean Architecture
  - Composite
  - Decorator
  - Strategy
- Ahora queremos manejar el ciclo de vida del Pedido.

Problema de diseño

- Antes podríamos hacer

```
if(estado.equals("CREADO")) {
   ...
} else if(...) {
   ...
}
```

- Problemas:
  - mucho if/else
  - difícil mantener
  - reglas dispersas

Patrón State

- Del pedido con condicionales al pedido que delega comportamiento en estados concretos.



STATE MACHINE

Objetivo de la clase

1. Punto de partida

Veremos por qué el ciclo de vida de un pedido suele terminar lleno de if/else.

2. Diseño con State

Separaremos el comportamiento variable en objetos de estado concretos.

3. Aplicación real

Mostraremos cómo quedó aplicado en Pedido, MagicCafeService y JPA.



El pedido cambia de comportamiento

- Problema
  - El pedido atraviesa un flujo: se crea, se paga, se prepara, se entrega o se cancela.
  - Cada acción tiene reglas distintas según el estado actual.
- Misma clase, reglas diferentes



CREADO

puede pagar
puede cancelar
puede agregar ítems

PAGADO

puede preparar
puede cancelar
no agrega ítems

ENTREGADO

no puede pagar
no puede cancelar
no puede cambiar

Antes: una clase Pedido llena de condicionales

Pedido.java · versión problemática

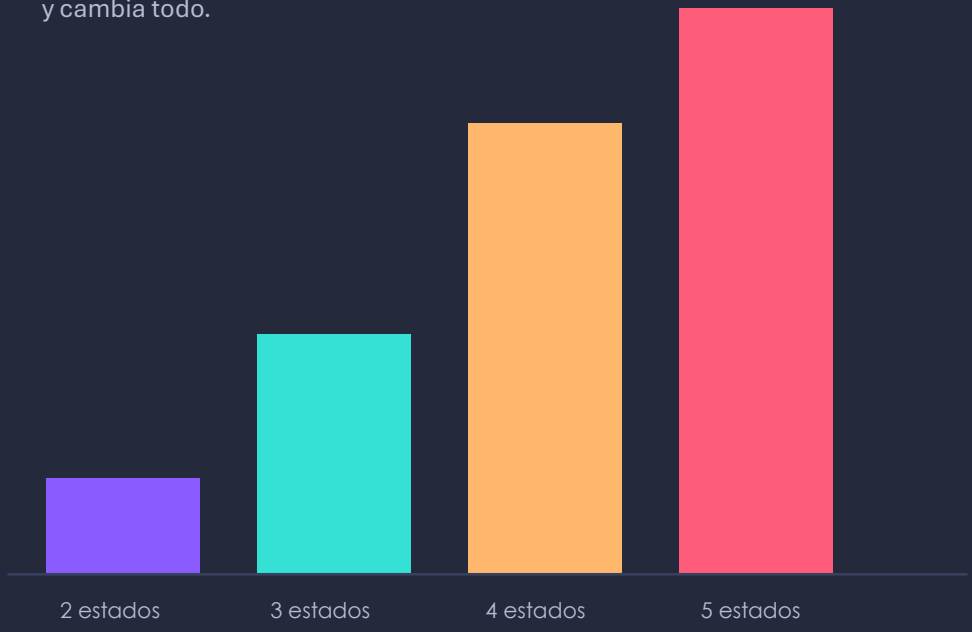
```
public void pagar() {
  if (estado.equals("CREADO")) {
    estado = "PAGADO";
  } else {
    throw new IllegalStateException(...);
  }
}

public void preparar() {
  if (estado.equals("PAGADO")) {
    estado = "PREPARANDO";
  } else {
    throw new IllegalStateException(...);
  }
}
```

Cuando agregamos acciones o estados...

Bad smell

Pedido se empieza a convertir en una clase que valida todo, decide todo y cambia todo.



Complejidad concentrada en Pedido

La idea central del patrón State

- Mover el comportamiento variable a objetos que representan estados.
- Pedido ya no pregunta todo el tiempo “¿en qué estado estoy?”.
- Le delega la acción al estado actual.



Aplicado a Magic Café

- No "meter State" por que sí, sino detectar el punto de cambio y aislarlo.

1. **Identificar estados**

   CREADO, PAGADO, PREPARANDO, ENTREGADO, CANCELADO.

2. **Definir acciones**

   pagar(), preparar(), entregar(), cancelar().

3. **Crear interfaz State**

   EstadoPedido expone el contrato común.

4. **Crear estados concretos**

   Cada estado permite solo transiciones válidas.

5. **Delegar desde Pedido**

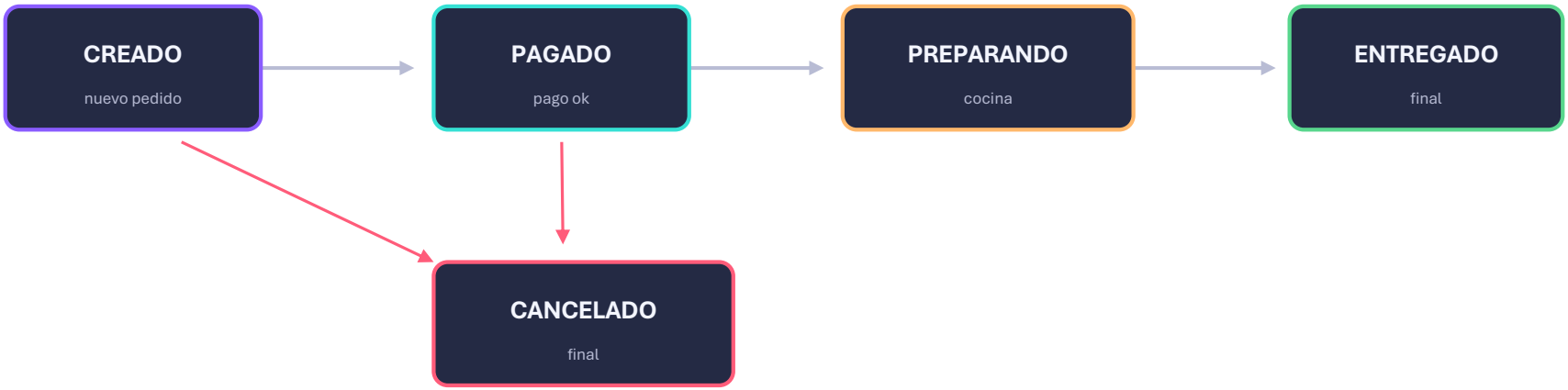
   Pedido llama al estado actual.

6. **Persistir estado**

   DB guarda String; Java reconstruye objeto State.

Paso 1: Identificar estados y transiciones

- El ciclo de vida del pedido se modela como una máquina de estados



Transiciones positivas

CREADO → PAGADO → PREPARANDO → ENTREGADO

Transiciones de corte

CREADO/PAGADO → CANCELADO

Estados finales

ENTREGADO y CANCELADO no cambian

Paso 2: Crear el contrato EstadoPedido

- Todas las acciones posibles quedan definidas en una interfaz común

domain/model/state/EstadoPedido.java

```
public interface EstadoPedido {

    String getNombre();

    void pagar(Pedido pedido);
    void preparar(Pedido pedido);
    void entregar(Pedido pedido);
    void cancelar(Pedido pedido);
}
```

¿Por qué una interfaz?

Porque Pedido no debe depender de clases concretas como EstadoCreado o EstadoPagado.

Polimorfismo

Pedido ejecuta estadoActual.pagar(this), pero cada estado decide qué hacer.

Protected Variations

El punto variable del sistema queda protegido detrás de EstadoPedido.

Paso 3: Crear una clase base para transiciones inválidas

- Los estados concretos sobreescriben solo lo que está permitido.

domain/model/state/EstadoPedidoBase.java

```
public abstract class EstadoPedidoBase implements
EstadoPedido {

  public void pagar(Pedido pedido) {
    throw new IllegalStateException(
      "No se puede pagar desde " + getNombre());
  }

  public void preparar(Pedido pedido) {
    throw new IllegalStateException(
      "No se puede preparar desde " + getNombre());
  }

  public void entregar(Pedido pedido) { ... }
  public void cancelar(Pedido pedido) { ... }
}
```

Por defecto

todo está prohibido

En cada estado

se habilita lo que corresponde

Resultado

menos duplicación y reglas claras

Paso 4: implementar estados concretos

- Cada estado conoce sus transiciones válidas

EstadoCreado.java

```
public class EstadoCreado extends EstadoPedidoBase {
  public String getNombre() { return "CREADO"; }

  public void pagar(Pedido pedido) {
    pedido.cambiarEstado(new EstadoPagado());
  }

  public void cancelar(Pedido pedido) {
    pedido.cambiarEstado(new EstadoCancelado());
  }
}
```

EstadoPagado.java

```
public class EstadoPagado extends EstadoPedidoBase {
  public String getNombre() { return "PAGADO"; }

  public void preparar(Pedido pedido) {
    pedido.cambiarEstado(new EstadoPreparando());
  }

  public void cancelar(Pedido pedido) {
    pedido.cambiarEstado(new EstadoCancelado());
  }
}
```

Pedido delega el comportamiento

- Pedido deja de validar todas las transiciones y delega en estadoActual

domain/model/Pedido.java

```
@Transient
private EstadoPedido estadoActual = new EstadoCreado();

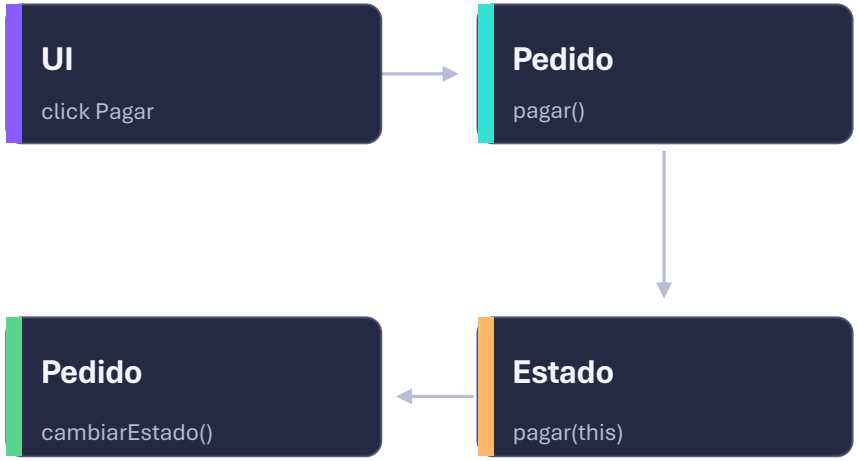
public void pagar() {
    estadoActual.pagar(this);
}

public void preparar() {
    estadoActual.preparar(this);
}

public void entregar() {
    estadoActual.entregar(this);
}

public void cancelar() {
    estadoActual.cancelar(this);
}

public void cambiarEstado(EstadoPedido nuevoEstado) {
    this.estadoActual = nuevoEstado;
    this.estado = nuevoEstado.getNombre();
}
```



Nota

El contexto conserva datos del pedido; el State decide la transición.

El service orquesta el caso de uso

- La capa de aplicación no conoce las reglas internas del estado

application/usecase/MagicCafeService.java

```
public void pagarPedido(Long pedidoId) {
    Pedido pedido = buscarPedido(pedidoId);
    pedido.pagar();
    pedidoRepository.save(pedido);
}

public void prepararPedido(Long pedidoId) {
    Pedido pedido = buscarPedido(pedidoId);
    pedido.preparar();
    pedidoRepository.save(pedido);
}

public void cancelarPedido(Long pedidoId) {
    Pedido pedido = buscarPedido(pedidoId);
    pedido.cancelar();
    pedidoRepository.save(pedido);
}
```



Resolver persistencia con JPA

- El comportamiento State vive en memoria; la base guarda un valor simple.
- La DB no necesita conocer clases Java. Guarda “PAGADO”; al cargar, Java reconstruye EstadoPagado.

Pedido.java

```
@Column(nullable = false)
private String estado = "CREADO";

@Transient
private EstadoPedido estadoActual = new EstadoCreado();

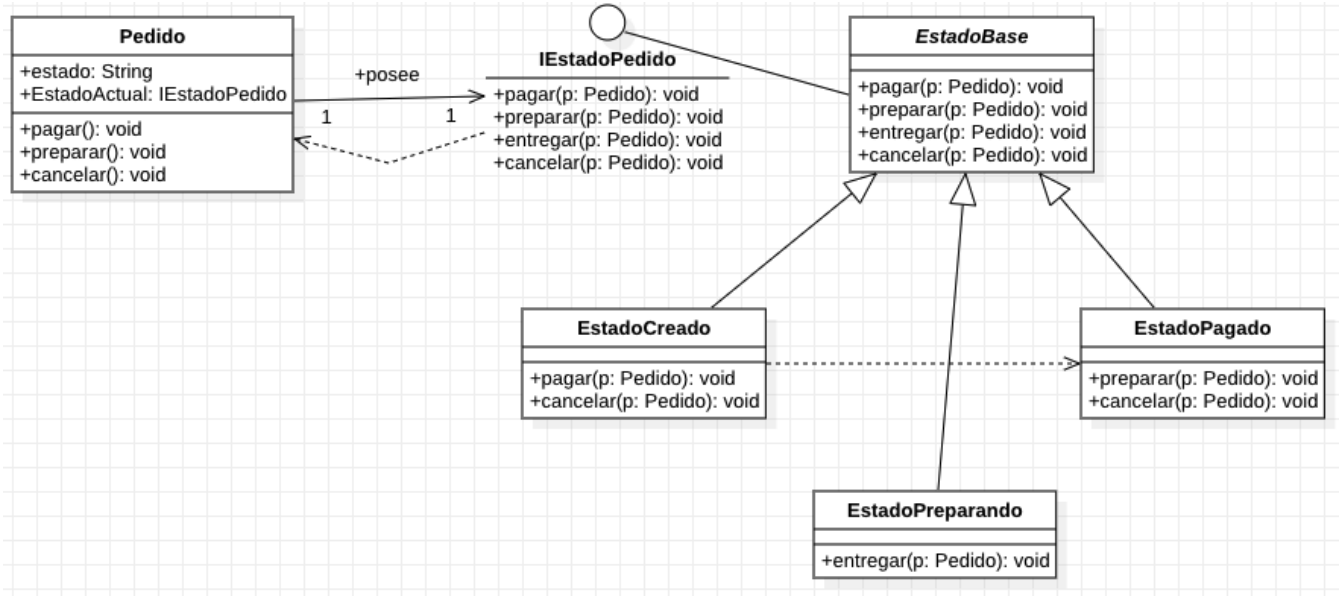
@PostLoad
private void reconstruirEstadoActual() {
  this.estadoActual = EstadoPedidoFactory
      .desdeNombre(this.estado);
}
```

EstadoPedidoFactory.java

```
return switch (nombre) {
  case "CREADO" -> new EstadoCreado();
  case "PAGADO" -> new EstadoPagado();
  case "PREPARANDO" -> new EstadoPreparando();
  case "ENTREGADO" -> new EstadoEntregado();
  case "CANCELADO" -> new EstadoCancelado();
  default -> throw new IllegalArgumentException(...);
};
```

Diagrama UML

- Vista UML simplificada de las clases intervinientes
- Pedido tiene una referencia a EstadoPedido. Cada estado concreto implementa el contrato y define las transiciones permitidas.



Antes y después

ANTES

Pedido concentra la lógica.

- muchos if/else o switch
- difícil agregar estados
- mayor riesgo de romper reglas
- baja cohesión en Pedido
- OCP débil



State

DESPUÉS

Pedido delega en estadoActual.

- reglas distribuidas por estado
- agregar estado es más simple
- transiciones inválidas centralizadas
- alta cohesión por clase
- OCP más fuerte

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

